A Handwritten Music Recognition System Based on Visual Object Detection and Semantic Assembly

Yuguang Yao Hok Seng

December 14, 2025

Introduction & Motivation

What is Optical Music Recognition (OMR)?

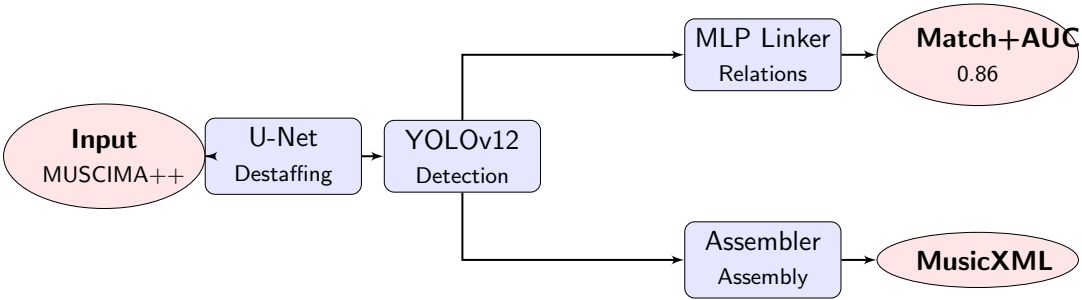
- Computationally read music notation in documents
- Invert the music encoding process to recover musical semantics
- Goal: Enable replayability and score editing

Why is OMR Challenging?

- **2D Structure**: Unlike OCR's linear text sequences
- **Context-Dependent**: Symbol meaning depends on spatial relationships
- **Size Variability**: From tiny dots to large braces
- **Complex Alignment**: Overlapping beams, slurs, and symbols

Project Goal: End-to-end pipeline from score image $\rightarrow$ MusicXML

System Pipeline Overview



Four Main Modules

U-Net → YOLO → MLP → Assembler

Module 1: Data Preprocessing (U-Net)

U-Net Architecture

- Encoder-decoder with skip connections
- Pixel-wise classification
- Separates symbols from staff lines

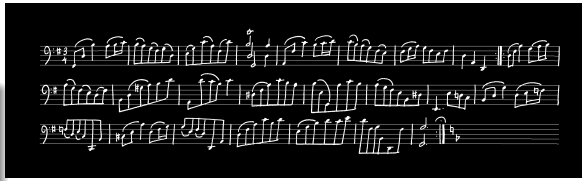


Figure: Original input

Preprocessing Pipeline

1. **Staff Removal**: Destaff to improve SNR
2. **Image Cropping**: 640×640 patches
3. **Annotation Conversion**: XML $\rightarrow$ YOLO format

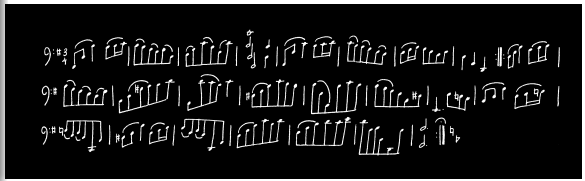


Figure: After U-Net destaffing

Module 2: Visual Detection (YOLOv12)

YOLO Configuration

- **Model**: YOLOv12-Large
- **Classes**: 117 symbol types
- **Training**: 100 epochs, SGD optimizer
- **Input Size**: 640×640 pixels
- **Augmentation**: Mosaic for robustness

Symbol Categories

Noteheads, Stems, Beams, Flags, Rests, Accidentals, Clefs, Time Signatures, Barlines, Tuplet Brackets, etc.

Performance

High mAP on common symbols $\rightarrow$ Solid foundation for assembly

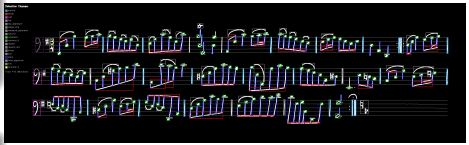


Figure: YOLOv12 detection results

Module 3: Relation Prediction (MLP Linker)

Problem Formulation

Link prediction on fully connected graph: each detected symbol = node

Feature Engineering

Geometric Features:

- Relative distance: d_x, d_y
- Normalized bbox coordinates
- IoU overlap ratio

Semantic Features:

- Class embeddings: $\mathbf{e}_u, \mathbf{e}_v \in \mathbb{R}^{32}$
- Concatenated: $[\mathbf{e}_u; \mathbf{e}_v] \in \mathbb{R}^{64}$
- Captures class-specific patterns

MLP Architecture

- 3 hidden layers with ReLU
- Output: $P(u \to v) \in [0, 1]$
- Constructs **Notation Graph**

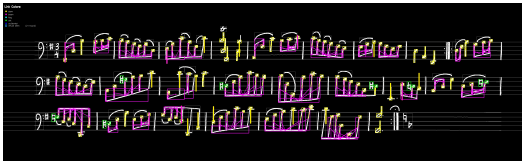


Figure: Notation graph with relationships

MLP Evaluation & Results

Match+AUC Metric

Combines two complementary aspects:

- **Match Score**: Precision of predicted edges vs. ground truth
- **AUC**: Ranking quality of edge probabilities

Match+AUC = 0.86

Successfully filters valid connections from quadratic pairs

Key Achievement

MLP effectively learns geometric and semantic patterns of symbol relationships, enabling robust notation graph construction.

Module 4: Semantic Assembly

Six-Phase Rule-Based Assembly Pipeline

1. **System & Staff Grouping**: Organize staves into musical rows
2. **Measure Slicing**: Horizontal segmentation via barline detection
3. **Symbol Assembly**: Link noteheads → stems → beams → dots
4. **Attribute Recognition**: Detect clef, key signature, time signature
5. **Tuplet Detection**: Cascaded rules (Bracket → Beam → Numeral)
6. **Pitch Determination**: Convert geometric positions to musical pitches

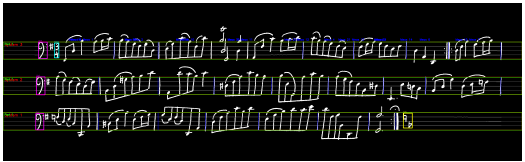


Figure: System structure

Key Features

- State persistence for attributes
- Virtual stem creation
- Priority-based rule ordering

Technical Challenge: Tuplet Detection

Cascaded Rule Set (Decreasing Constraint Strength)

1. **Rule 1 - Bracket-Based** [Highest Priority]
   - Search for `tuple_bracket` covering exactly 3 notes
   - Relaxed: accepts without `numeral_3` if 3 notes present
2. **Rule 2 - Beam-Based** [Second Priority]
   - Beams linking 3k stems with nearby `numeral_3`
   - **Exclusion**: Finger numbers (vertically aligned above single note)
3. **Rule 3 - Numeral-Centered** [Lower Priority]
   - Find 3 closest notes to isolated `numeral_3`
   - Exclude time signature numerals
4. **Rule 4 - Duration Modification**
   - Apply: $duration_{xml} = base\_duration \times \frac{2}{3}$
5. **Rule 5 - Sanity Check** [Fallback]
   - Detect measure overflow, search for 3 identical-duration notes

Results & MusicXML Output

Achievements

- **End-to-End**: Image → MusicXML
- **Compatible**: Opens in MuseScore
- **Polyphonic**: Handles multiple voices
- **Complex Rhythms**: Tuplets, dotted notes

Future Improvements

1. **Enhanced Linker**: Small models for sub-processes
2. **More Symbol Types**: Expand coverage
3. **Interactive Processing**: User-modifiable probabilities

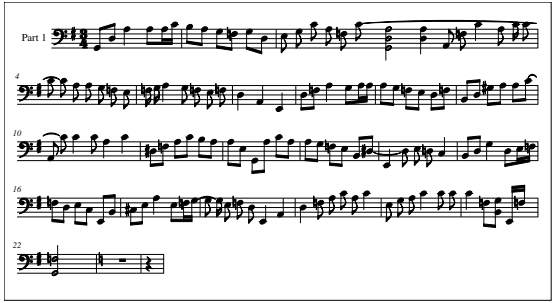


Figure: Final MusicXML output rendered as musical notation

Thank You!